PLANO DE DESENVOLVIMENTO

Chatbot de Suporte Técnico - Hikvision

Divisão de tarefas para 5 integrantes • Python • Git/GitHub • Commits individuais

Este plano adapta a mesma lógica de divisão e versionamento do projeto de Gestão de Estoque para o projeto de IA de dúvidas técnicas de equipamentos Hikvision. Cada integrante trabalha em sua própria branch, registra entregas em commits pequenos e objetivos e, ao final, abre Pull Request diretamente para main. Não será utilizada branch develop.

1. Estrutura prevista do projeto

```
A3-CHATBOT-IT/
├── data/
│   ├── base_conhecimento.json      # Base técnica curada Hikvision
│   └── test_cases.csv              # 20 perguntas obrigatórias de validação
├── src/
│   ├── __init__.py
│   ├── preprocessor.py             # Limpeza e normalização do texto
│   ├── retriever.py                # TF-IDF + similaridade de cosseno
│   ├── engine.py                   # Threshold, decisão e fallback
│   └── metrics.py                  # Acurácia e métricas de validação
├── tests/
│   ├── test_preprocessor.py
│   ├── test_retriever.py
│   └── test_engine.py
├── app.py                          # Interface Streamlit
├── validar_base.py                 # Validação da base técnica
├── requirements.txt
├── README.md
└── .gitignore
```

Estado atual visível no GitHub: o repositório já possui data/, .gitignore, README.md, requirements.txt e validar_base.py. O plano abaixo parte desse ponto e distribui apenas as entregas necessárias para completar o projeto.

2. Organização das branches

```
main
├── feature/base-preprocessamento
├── feature/retriever-tfidf
├── feature/engine-fallback
├── feature/interface-streamlit
└── feature/testes-metricas
```

Fluxo inicial recomendado para todos:

```
git checkout main
git pull origin main
git checkout -b feature/NOME-DA-FEATURE
```

PESSOA 1 - Base de conhecimento e pré-processamento

Branch:

```
git checkout main
git pull origin main
git checkout -b feature/base-preprocessamento
```

Commit 1 - Estrutura da base Hikvision

- Arquivos: data/base_conhecimento.json
- Organizar a base técnica nas três intenções do projeto: Configuração/Instalação, Erros/Falhas e Segurança/Senhas.
- Padronizar os campos necessários para pergunta, resposta, categoria/intenção e referência técnica.

```
git add data/base_conhecimento.json
git commit -m "feat: estrutura base de conhecimento Hikvision"
```

Commit 2 - Validação da base

- Arquivos: validar_base.py
- Validar JSON, campos obrigatórios, textos vazios, categorias inválidas e possíveis duplicidades.
- Aproveitar e evoluir o validar_base.py que já existe no repositório.

```
git add validar_base.py
git commit -m "feat: adiciona validacao da base de conhecimento"
```

Commit 3 - Pré-processamento de texto

- Arquivos: src/__init__.py e src/preprocessor.py
- Implementar normalização para minúsculas, remoção de acentos, limpeza de caracteres especiais e espaços excedentes.
- Disponibilizar uma função principal preprocessar() para ser usada pelo motor de busca.

```
git add src/__init__.py src/preprocessor.py
git commit -m "feat: implementa preprocessamento de texto"
```

Commit 4 - Testes do pré-processamento

- Arquivos: tests/test_preprocessor.py
- Testar acentuação, maiúsculas/minúsculas, pontuação, caracteres especiais e entrada vazia.

```
git add tests/test_preprocessor.py
git commit -m "test: adiciona testes do preprocessamento"
```

Envio da branch

```
git push -u origin feature/base-preprocessamento
```

PESSOA 2 - Motor TF-IDF e Similaridade

Branch:

```
git checkout main
git pull origin main
git checkout -b feature/retriever-tfidf
```

Commit 1 - Carregamento da base

- Arquivos: src/retriever.py
- Implementar a leitura de data/base_conhecimento.json e preparar os textos para o motor de recuperação.

```
git add src/retriever.py
git commit -m "feat: implementa carregamento da base de conhecimento"
```

Commit 2 - Vetorização TF-IDF

- Arquivos: src/retriever.py
- Utilizar TfidfVectorizer do scikit-learn para transformar a base técnica em vetores.
- Usar o pré-processamento definido no projeto antes da vetorização.

```
git add src/retriever.py
git commit -m "feat: implementa vetorizacao tfidf"
```

Commit 3 - Similaridade de cosseno

- Arquivos: src/retriever.py
- Calcular a similaridade entre a pergunta do usuário e os registros da base usando cosine_similarity.
- Retornar resposta, categoria/intenção e valor de similaridade.

```
git add src/retriever.py
git commit -m "feat: implementa busca por similaridade de cosseno"
```

Commit 4 - Seleção do melhor resultado

- Arquivos: src/retriever.py
- Selecionar automaticamente o registro com maior similaridade e disponibilizar o resultado para o engine.

```
git add src/retriever.py
git commit -m "feat: seleciona melhor resposta da base"
```

Commit 5 - Testes do motor de recuperação

- Arquivos: tests/test_retriever.py
- Testar perguntas idênticas, pequenas reformulações e situações sem correspondência forte.

```
git add tests/test_retriever.py
git commit -m "test: adiciona testes do motor tfidf"
```

Envio da branch

```
git push -u origin feature/retriever-tfidf
```

PESSOA 3 - Engine, confiança e fallback

Branch:

```
git checkout main
git pull origin main
git checkout -b feature/engine-fallback
```

Commit 1 - Engine principal

- Arquivos: src/engine.py
- Criar o fluxo que recebe a pergunta, chama o pré-processador e solicita a melhor resposta ao retriever.

```
git add src/engine.py
git commit -m "feat: implementa engine principal do chatbot"
```

Commit 2 - Limiar de confiança

- Arquivos: src/engine.py
- Implementar o threshold do projeto para decidir se a resposta técnica possui confiança suficiente.
- Padronizar o valor inicial em 0.40, conforme a regra principal da especificação.

```
git add src/engine.py
git commit -m "feat: implementa limiar de confianca"
```

Commit 3 - Fallback / abstenção

- Arquivos: src/engine.py
- Se a similaridade ficar abaixo do threshold, não inventar resposta e acionar a mensagem padrão de encaminhamento humano.

```
git add src/engine.py
git commit -m "feat: implementa fallback para baixa confianca"
```

Commit 4 - Controle de escopo

- Arquivos: src/engine.py
- Tratar consultas fora das três intenções previstas e encaminhá-las para fallback.

```
git add src/engine.py
git commit -m "feat: implementa controle de escopo do chatbot"
```

Commit 5 - Testes do engine

- Arquivos: tests/test_engine.py
- Testar confiança alta, confiança baixa, pergunta ambígua, fora do escopo e comportamento do fallback.

```
git add tests/test_engine.py
git commit -m "test: adiciona testes de threshold e fallback"
```

Envio da branch

```
git push -u origin feature/engine-fallback
```

PESSOA 4 - Interface Streamlit

Branch:

```
git checkout main
git pull origin main
git checkout -b feature/interface-streamlit
```

Commit 1 - Interface inicial

- Arquivos: app.py
- Criar app.py com título, apresentação do chatbot e campo para receber perguntas do usuário.

```
git add app.py
git commit -m "feat: cria interface inicial do chatbot"
```

Commit 2 - Integração da interface com o engine

- Arquivos: app.py
- Enviar a pergunta recebida pelo Streamlit para processar_pergunta() e apresentar a resposta devolvida pelo engine.

```
git add app.py
git commit -m "feat: integra interface com engine do chatbot"
```

Commit 3 - Histórico de conversa

- Arquivos: app.py
- Exibir mensagens do usuário e do chatbot em formato de conversa durante a sessão.

```
git add app.py
git commit -m "feat: adiciona historico de conversa no streamlit"
```

Commit 4 - Exibição do fallback e confiança

- Arquivos: app.py
- Apresentar mensagem clara quando a confiança for insuficiente e indicar encaminhamento para suporte humano.

```
git add app.py
git commit -m "feat: exibe fallback e nivel de confianca"
```

Commit 5 - Tratamento de erros da interface

- Arquivos: app.py
- Evitar quebra da aplicação em pergunta vazia, base ausente, JSON inválido ou erro de carregamento dos módulos.

```
git add app.py
git commit -m "fix: adiciona tratamento de erros da interface"
```

Envio da branch

```
git push -u origin feature/interface-streamlit
```

PESSOA 5 - Massa de testes, métricas e documentação

Branch:

```
git checkout main
git pull origin main
git checkout -b feature/testes-metricas
```

Commit 1 - Massa de 20 perguntas

- Arquivos: data/test_cases.csv
- Criar 10 perguntas diretas, 4 reformulações semânticas, 3 perguntas ambíguas e 3 perguntas fora do escopo.
- Registrar categoria/intenção e resultado esperado para permitir validação automática.

```
git add data/test_cases.csv
git commit -m "test: adiciona massa de vinte perguntas para validacao"
```

Commit 2 - Execução automatizada dos testes

- Arquivos: src/metrics.py
- Criar metrics.py para ler test_cases.csv e executar cada pergunta utilizando o engine do chatbot.

```
git add src/metrics.py
git commit -m "feat: implementa execucao automatica dos testes"
```

Commit 3 - Métricas de avaliação

- Arquivos: src/metrics.py
- Calcular acurácia global, acertos, erros, falsos positivos, falsos negativos e comportamento da abstenção.

```
git add src/metrics.py
git commit -m "feat: adiciona metricas de avaliacao do chatbot"
```

Commit 4 - Atualização das dependências

- Arquivos: requirements.txt
- Garantir que requirements.txt contenha apenas as bibliotecas necessárias para o projeto: Streamlit, scikit-learn, pandas e numpy.

```
git add requirements.txt
git commit -m "chore: atualiza dependencias do projeto"
```

Commit 5 - README de execução e validação

- Arquivos: README.md
- Documentar instalação, execução do Streamlit, execução das métricas, estrutura de arquivos e objetivo do chatbot.

```
git add README.md
git commit -m "docs: finaliza instrucoes de instalacao testes e execucao"
```

Envio da branch

```
git push -u origin feature/testes-metricas
```

3. Histórico esperado de commits

PESSOA 1

- feat: estrutura base de conhecimento Hikvision
- feat: adiciona validacao da base de conhecimento
- feat: implementa preprocessamento de texto
- test: adiciona testes do preprocessamento

PESSOA 2

- feat: implementa carregamento da base de conhecimento
- feat: implementa vetorizacao tfidf
- feat: implementa busca por similaridade de cosseno
- feat: seleciona melhor resposta da base
- test: adiciona testes do motor tfidf

PESSOA 3

- feat: implementa engine principal do chatbot
- feat: implementa limiar de confianca
- feat: implementa fallback para baixa confianca
- feat: implementa controle de escopo do chatbot
- test: adiciona testes de threshold e fallback

PESSOA 4

- feat: cria interface inicial do chatbot
- feat: integra interface com engine do chatbot
- feat: adiciona historico de conversa no streamlit
- feat: exibe fallback e nivel de confianca
- fix: adiciona tratamento de erros da interface

PESSOA 5

- test: adiciona massa de vinte perguntas para validacao
- feat: implementa execucao automatica dos testes
- feat: adiciona metricas de avaliacao do chatbot
- chore: atualiza dependencias do projeto
- docs: finaliza instrucoes de instalacao testes e execucao

4. Regras para a avaliação pelo Git

- Não concentrar toda a atividade em um único commit.
- Cada commit deve representar uma funcionalidade, correção, teste ou documentação específica.
- Evitar mensagens genéricas como "minha parte", "alterações" ou "final".
- Preferir git add com os arquivos relacionados ao commit, em vez de git add . automaticamente.
- Cada integrante deve fazer commits usando sua própria conta/configuração do Git.
- Cada integrante deve fazer push da própria branch.
- Não realizar merge das branches de desenvolvimento antes da fase final de integração.
- Na fase final, cada branch deverá abrir Pull Request diretamente para main e ser revisada antes do merge.

5. Integração final diretamente na main

Como não haverá branch develop, a integração ocorrerá apenas ao final do desenvolvimento. Para reduzir conflitos, os merges devem ser realizados em uma ordem compatível com as dependências do projeto:

```
1. feature/base-preprocessamento
2. feature/retriever-tfidf
3. feature/engine-fallback
4. feature/interface-streamlit
5. feature/testes-metricas
```

Antes de abrir o Pull Request, o responsável pela branch deve atualizar sua branch com a main mais recente:

```
git checkout main
git pull origin main
git checkout feature/SUA-BRANCH
git merge main
# resolver conflitos, se existirem
git push
```

Depois, abrir no GitHub:

```
Pull Request: feature/SUA-BRANCH  ->  main
```

6. Teste final antes de concluir a main

- Executar a validação da base de conhecimento.
- Instalar as dependências com pip install -r requirements.txt.
- Executar os testes automatizados disponíveis.
- Executar python src/metrics.py e conferir o resultado das 20 perguntas.
- Executar streamlit run app.py e validar o fluxo completo do chatbot.
- Confirmar que perguntas fora do escopo e de baixa confiança acionam fallback em vez de resposta inventada.

7. Critério de entrega individual

- Pessoa 1: demonstrar base técnica válida e pré-processamento funcionando.
- Pessoa 2: demonstrar vetorização TF-IDF, similaridade de cosseno e recuperação do melhor resultado.
- Pessoa 3: demonstrar decisão por threshold, controle de escopo e fallback.
- Pessoa 4: demonstrar interface Streamlit recebendo pergunta e apresentando resposta/fallback.
- Pessoa 5: demonstrar test_cases.csv, execução automática, métricas e documentação de execução.
- Todos: entregar código versionado, commits individuais, evidências de execução e explicação das decisões técnicas.

8. Branch para publicar este plano de desenvolvimento

Para versionar o PDF sem misturá-lo às branches de código, crie uma branch exclusiva de documentação. Esta branch pode ser enviada ao GitHub agora, mesmo que as branches de desenvolvimento ainda não tenham sido concluídas.

Nome recomendado da branch:

```
docs/divisao-desenvolvimento
```

Comandos para criar a branch:

```
git checkout main
git pull origin main
git checkout -b docs/divisao-desenvolvimento
```

Coloque o PDF dentro do repositório. Sugestão de caminho:

```
docs/Plano_Desenvolvimento_Chatbot_Hikvision_5_Pessoas.pdf
```

Se a pasta docs ainda não existir:

```
mkdir docs
```

Depois de copiar o PDF para a pasta docs, versionar o arquivo:

```
git status
git add docs/Plano_Desenvolvimento_Chatbot_Hikvision_5_Pessoas.pdf
git commit -m "docs: adiciona plano de divisao de desenvolvimento"
git push -u origin docs/divisao-desenvolvimento
```

Resultado esperado no GitHub:

```
main
├── docs/divisao-desenvolvimento      # branch deste documento
├── feature/base-preprocessamento
├── feature/retriever-tfidf
├── feature/engine-fallback
├── feature/interface-streamlit
└── feature/testes-metricas
```

Importante: a branch docs/divisao-desenvolvimento serve apenas para guardar e compartilhar o plano. Ela não substitui nenhuma das cinco branches de desenvolvimento e não precisa ser usada para código do chatbot.